

Resilience

A hands-on course in building agents by building their harness

Lesson 8

Everything so far assumed the other side answers

Lesson 7 can tell you exactly where a run broke. Over a long run, this is what breaks first:

- the API drops a call
- the model is overloaded
- a response is cut off in the middle of a command
- the process dies halfway through, and the work goes with it

Keep going, or stop somewhere you can start again

Two jobs:

- **Get through the failures that pass**
- **Leave a state you can pick up from** when one doesn't, so the hour before it isn't lost

Built on the **model interface** and **output**, where the harness reaches out and the world gets to say no. To pick a run back up it **reads context's own record**: Lesson 4's episode.

Not every failure means the same thing

Worth trying again	Not worth trying again
the network dropped, or a timeout	a bad API key (401)
"slow down" (429)	a request the API won't accept (400)
any 5xx, and "overloaded" (529)	a model that doesn't exist (404)
<i>the same request will probably work</i>	<i>it gets the same answer every time</i>

The SDK retries; quark adds a backup model

How to retry: back off (one second, two, four), add jitter, wait as long as the server says, and stop after a few tries.

- The Anthropic SDK already does all of this, set by `max_retries`
- So quark doesn't write a retry loop: it turns the setting up and sets a timeout
- What the SDK can't do is choose a different model
- When nothing answers, stop with a message, not a stack trace

Every request gets an answer the model can read

A command can fail in ways the harness doesn't see coming:

- it hangs (the sandbox already puts a time limit on that)
- it prints bytes that aren't text
- the request arrives cut off by the token limit, half a command

This was cut off, it wasn't run, send it again shorter is something the model can act on. A crash isn't.

The trap: a command that may already have run

- The episode has the model's request on disk *before* the command runs
- If the harness died mid-command, the record ends with a request and no result
- The API won't accept that, and the harness can't just run it again: it might have been `git push`
- So the missing result becomes an error: *interrupted; may or may not have run; check before repeating it*
- The model, which knows what the command was for, looks and decides

`call()` grows a backup

Lesson 7's `quark.py` plus 29 lines. In `# — model interface —` (abridged):

```
client = Anthropic(timeout=300, max_retries=3)
MODELS = ["claude-sonnet-5-5", "claude-opus-5-5"]
class Down(Exception): pass
def call(**request):
    for model in MODELS:
        try: return client.messages.create(model=model, **request)
        except (APIConnectionError, APIStatusError) as e:
            if ... e.status_code < 500 and e.status_code != 429: raise
            trace(event="model_failed", model=model, error=type(e).__name__)
    raise Down()
```

`unfinished()` reads the last episode back

In `# — context —`, after `add()` (abridged):

```
def unfinished():
    episodes = sorted(glob.glob(".quark/episodes/*.jsonl"))
    if not episodes: return None
    messages = [json.loads(line) for line in open(episodes[-1])]
    last = messages[-1]
    if last["role"] == "assistant" and not any(... "tool_use" ...): return None
    if read(f"unfinished session: ... pick it up? [y/N] ").lower() != "y": return None
    return episodes[-1], messages
```

In `# — input —`, it's asked before anything else is read:

```
resumed = unfinished()
input = "" if resumed else " ".join(sys.argv[1:]) or read("> ")
```

A resumed session continues in its own episode

In # — control flow — (abridged):

```
if resumed:
    EPISODE, working_memory = resumed
    if working_memory[-1]["role"] == "assistant":
        add(working_memory, {"role": "user", "content": [...]}))
else:
    add(working_memory, {"role": "user", "content": input})
trace(event="start", input=input, resumed=bool(resumed))
```

Each unanswered request gets: *"interrupted: the harness stopped before this finished, so it may or may not have run. Check before repeating it."*

Stop with a sentence, and never run half a command

```
except Down:  
    sys.exit("[the model isn't answering. ...]")
```

```
cmd = block.input.get("cmd")  
print(f"$ {cmd}")  
if not cmd or (response.stop_reason == "max_tokens" and ...):  
    print("[cut off, not run]")  
    input.append({"type": "tool_result", ..., "is_error": True})  
    continue
```

(abridged) Cut off means `max_tokens` and this is the last block. The result says:
"your request was cut off at the token limit, so it was not run. Send it again, shorter."
And `errors="replace"` turns bytes that aren't text into .

Run it: the preferred model is down

A stand-in for the API answers "529 overloaded" to every request for `claude-sonnet-5-5`, and forwards the rest:

```
$ wc -c notes.txt
6 notes.txt

notes.txt is 6 bytes.
```

The output looks like any other run. The trace doesn't:

```
{"event": "model_failed", "model": "claude-sonnet-5-5", "error": "OverloadedError"}
{"event": "model", "seconds": 5.22, "stop_reason": "tool_use"}
```

(shortened)

Run it: nothing answers, then the API comes back

Every request fails, for every model. quark stops with:

[the model isn't answering. Everything so far is in the episode; run quark again to pick it up]

Run `quark.py` again with no input and answer `y` (shortened):

```
notes.txt is 6 bytes.
```

```
{"event": "start", "input": "", "resumed": true}
```

Run it: the harness is killed mid-command

The task is one slow command, `sleep 15 && echo finished > flag.txt`. The harness is killed with `kill -9` on its PID twelve seconds in.

- The episode was written when the model asked for the command, before it ran
- `flag.txt` isn't there, and then it is: the container wasn't killed, so the command finished in it
- `atexit` can't run after `kill -9`
- The command did run, and the harness doesn't know

Resumed, the model checks before repeating

```
finished
```

```
`flag.txt` contains `finished`.
```

(shortened) It ran `ls -l flag.txt; cat flag.txt`, then said:

The first run of the command was interrupted, and I wasn't sure it had completed. I checked the file before running it again. The file already existed and held that text, so I didn't repeat the command.

What else resilience can be

- **Retries and waits:** backoff with jitter and a cap, `Retry-After`, or a limiter that keeps under the rate limit
- **The backup:** another model, provider or region; a switch pays the full prompt once, since the cache is per model
- **Remembering a failure:** a *circuit breaker* leaves a failed model alone for a minute
- **Streaming and tools:** keep what arrived, time out on silence; kill the process group, cap output
- **State and restarts:** saved every step, as an event log, restarted by you, a supervisor or a service

Products, and a fuller example

Gateways like LiteLLM and OpenRouter handle retries, fallbacks and rate limits. Durable execution (Temporal) and checkpointing (LangGraph) pick a crashed process up at its step.

`resilience.py`, built on Lesson 3's loop, does the retries itself, in plain sight:

- `retryable()` sorts failures; `pause()` honors `retry-after`, else backs off with jitter
- `ask()` benches a model that gave up for `BENCH` seconds
- `run()` kills the whole process group and caps output
- `save()` checkpoints every step; `resilience.py resume` picks it up

The preferred model down, with a circuit breaker

```
[claude-sonnet-5-5: OverloadedError, try 1 of 4, waiting 0.7s]
[claude-sonnet-5-5: OverloadedError, try 2 of 4, waiting 1.7s]
[claude-sonnet-5-5: OverloadedError, try 3 of 4, waiting 2.5s]
[claude-sonnet-5-5: OverloadedError, giving up on it for 60s]
[answered by the backup, claude-opus-5-5]
$ wc -c notes.txt
6 notes.txt
[answered by the backup, claude-opus-5-5]
notes.txt is 6 bytes.
[done in 2 steps]
```

Waits grow with jitter: 0.7, 1.7, 2.5. The second call skipped the benched model: six requests, where quark sent ten.

The rule

Assume the call can fail, and make failure something the harness handles instead of something that ends the run.

- Retry what passes, a bounded number of times
- Have somewhere else to go, and stop with a sentence when everywhere is out
- Answer every tool request, even a broken one, with a result the model can read
- Keep a record written before each step, so what was in flight is reported as *unknown*

What resilience never does

- **Context:** it reads the record context already keeps; it adds no store of its own
- **Control flow:** same loop, same stops; a retry is inside one call, and the only new way out is `Down`
- **Input:** untouched, apart from one question at startup, asked with the same `read()`
- **What the model sees:** one new message, the "interrupted" one, written like any other result

What's missing: alive, but not fast or cheap

- Every call re-sends the whole conversation and waits for the whole reply
- A command that prints a megabyte puts a megabyte in the next request
- Three commands that could run side by side wait for each other
- Retries and fallbacks cost time that only the trace shows

Making each step smaller, faster and cheaper is its own layer.

Next: Performance

Lesson 9